

Pañuelo al Viento

Documentación de sistema

18 · Guía de incorporación

Versión del producto **0.1.0+1**
Commit **6efae74** · 2026-08-25
Rama **main**

Generado desde los Markdown de docs/system-documentation/ con tool/build_docs_pdf.py.
La versión y el commit se leen del repositorio; no están escritos a mano.

Contenido de este documento

Antes de nada: tres cosas que sorprenden

La primera hora

El primer día

La primera semana

Preguntas frecuentes de quien llega

Continuar por

18 · Guía de incorporación

Itinerario para quien llega hoy al proyecto. Está pensado para leerse en orden y en tres tramos: la primera hora, el primer día y la primera semana.

Antes de nada: tres cosas que sorprenden

Las tres cuestan tiempo si nadie las dice.

android/ y windows/ no existen. Están en `.gitignore` a propósito. Se generan con `tool/bootstrap.ps1` o `tool/bootstrap.sh`. Si ejecutas `flutter run` sin eso, no hay dispositivos.

La versión de Flutter importa. CI fija 3.44.6 y ejecuta `dart format --set-exit-if-changed`. El formateador de Dart cambia entre versiones: con otra, el formato falla aunque tu código sea correcto.

Buena parte de la verificación no necesita Flutter. Tres validadores en Node comprueban currículum, coherencia del repositorio y landing en segundos. Empieza siempre por ahí.

La primera hora

1 · Poner el proyecto en marcha (20 min)

```
.tool/bootstrap.ps1 # Windows
```

```
./tool/bootstrap.sh # Linux y macOS
```

El script genera las plataformas, aplica la identidad del producto, descarga dependencias y ejecuta las cuatro compuertas. Aborta al primer fallo, así que si termina, el proyecto está sano.

```
flutter run -d windows
```

2 · Usar la aplicación (15 min)

No leas código todavía. Recorre las cuatro secciones, abre una clase, marca las tres casillas, complétala y comprueba que el porcentaje cambia. Abre el laboratorio de ritmo y cambia entre 3+3 y 2+2+2. Reinicia el avance.

En quince minutos tendrás el modelo mental que hace falta para que el código se lea solo.

3 · Leer dos documentos (20 min)

- [../README.md](#) — qué es el producto y qué promete.
- [01 · Visión general](#) — qué hace, qué no hace y con qué decisiones.

4 · Ejecutar las compuertas (5 min)

```
node tool/validate_repository.mjs
node tool/validate_curriculum.mjs
flutter test
```

Verás Repositorio válido: 32 archivos esenciales..., Currículo válido: 8 niveles, 24 clases y 72 actividades. y All tests passed! con 25 pruebas. Ese es el estado normal; si algo sale distinto, algo cambió.

El primer día

Leer el código en este orden

Son 2 417 líneas. Se leen en una mañana, y el orden importa porque cada archivo se apoya en el anterior.

#	Archivo	Líneas	Por qué aquí
1	lib/domain/curriculum.dart	193	Los cuatro modelos. Sin dependencias. Todo lo demás habla de estos objetos
2	assets/content/curriculum.json	—	Abre una clase entera y compárala con el modelo. Con eso entiendes el contenido
3	lib/data/curriculum_repository.dart	27	Trivial, pero cierra el círculo
4	lib/data/progress_repository.dart	68	Lee los comentarios. Explican por qué la interfaz existe
5	lib/state/app_state.dart	126	El archivo más importante. Todas las reglas de negocio
6	lib/main.dart	89	El arranque y la pantalla de recuperación
7	lib/screens/home_shell.dart	143	La navegación y el <code>IndexedStack</code> , que tiene una consecuencia que hay que conocer

8	<code>lib/screens/lesson_screen.dart</code>	299	El flujo con más reglas de producto
9	<code>lib/screens/rhythm_lab_screen.dart</code>	356	El único con lógica temporal. Lee <code>_scheduleNextPulse</code> despacio
10	El resto	—	Se leen solos

Después: [06 · Explicación profunda](#), que recorre esos mismos archivos por flujos en vez de por archivo.

Las cuatro reglas que no se rompen

Si te quedas con cuatro cosas de este día, que sean estas.

1. La persistencia se confirma antes de publicarse.

```
await _progressRepository.writeCompletedLessonIds(updated); // puede lanzar
_completedLessonIds = updated; // solo si no lanzó
notifyListeners();
```

Invertir ese orden produce el fallo más caro del producto: avance que se ve y desaparece al reabrir. Hay tres pruebas dedicadas a impedirlo.

2. `lib/domain` no importa nada. Ni Flutter. Eso es lo que permite validar el currículo sin levantar un entorno de widgets. Un import `'package:flutter/...'` ahí rompe tres pruebas y un validador.

3. La versión sale de `pubspec.yaml` y de ningún otro sitio. Todo lo demás la consume a través de `tool/app_version.mjs`. El CHANGELOG documenta el fallo que esa regla evita.

4. Cámara, micrófono e internet están fuera del producto. No por descuido: hay tres controles automáticos que bloquean la publicación si aparecen. Añadir una dependencia que los arrastre hará fallar la release.

Los comandos del día a día

```
flutter test # 25 pruebas, ~2 s
flutter analyze # ~3 s en caliente
dart format lib test tool # antes de commitear
dart format --output=none --set-exit-if-changed lib test tool # como lo ve CI
node tool/validate_repository.mjs
node tool/validate_curriculum.mjs
node tool/build_site.mjs --serve # landing en :8080
```

La primera semana

Documentos por orden de utilidad

Cuándo	Documento
Antes de tocar contenido	../CURRICULUM.md y ../PEDAGOGY_AND_SAFETY.md
Antes de tocar la interfaz	../ACCESSIBILITY.md

Antes de tocar datos o dependencias	../PRIVACY.md y ../PERMISSIONS.md
Antes de tocar la publicación	13 · Despliegue y ../RELEASE_CHECKLIST.md
Para saber qué está roto	15 · Riesgos
Cuando algo falle	14 · Troubleshooting
Cuando busques un símbolo	05 · Referencia técnica
Cuando busques dónde vive algo	04 · Mapa del código y 19 · Trazabilidad

Primeras tareas recomendadas

Elegidas para que toques una parte distinta del sistema en cada una. Todas están registradas como hallazgos en [15](#), así que sabrás si acertaste.

Tarea	Toca	Dificultad
Corregir el ROADMAP , que da por pendiente la landing ya publicada (R-03)	Documentación	Trivial
Unificar el nombre del nivel 8 entre el dato y dos documentos (R-04)	Contenido y documentación	Trivial
Sustituir «24 clases» escrito a mano por <code>state.totalCount</code> (R-06)	Interfaz y estado	Fácil
Añadir docs/CONTENT_PRODUCTION.md a los archivos esenciales (R-12)	Herramientas	Fácil
Añadir ramas de dibujo para <code>pair</code> y <code>free</code> (R-02)	Widgets y pintura	Media
Escribir la primera prueba del laboratorio de ritmo (R-11)	Pruebas y tiempo	Media-alta

La última es la más formativa: te obliga a entender la corrección de deriva, que es la pieza técnica más cuidada del proyecto.

Cómo se contribuye aquí

[../CONTRIBUTING.md](#) es la fuente autorizada. Lo esencial:

```
flutter pub get
node tool/validate_repository.mjs
node tool/validate_curriculum.mjs
dart format lib test tool
flutter analyze
flutter test
dart run tool/validate_curriculum.dart
```

Y siete requisitos de contenido que **no** son negociables: lenguaje comprensible a los diez años, no presentar una variante local como la única correcta, incluir adaptación cuando un movimiento pueda excluir a alguien, no añadir

medios sin licencia documentada, nada de publicidad ni rastreo, y actualizar las pruebas cuando cambie la estructura del currículum.

Si tu cambio toca la interfaz, regenera las capturas (`node tool/capture_screenshots.mjs`). Si toca el empaquetado de Android, comprueba el binario (`node tool/verify_apk.mjs <apk>`).

Preguntas frecuentes de quien llega

¿Dónde está el estado global? No hay. `main` crea un `AppState` y lo pasa a mano por los constructores. Con quince archivos es la opción correcta; ver [03](#) · [Arquitectura](#).

¿Por qué el laboratorio de ritmo no recibe `AppState`? Porque no comparte nada. Es la única pantalla con constructor `const`.

¿Por qué hay dos validadores de currículum? Uno en Node y otro en Dart, con las mismas reglas. Es una duplicación registrada como R-08; CI solo ejecuta el de Node.

¿Por qué el currículum está en JSON y no en Dart? Para poder editarlo y validarlo sin recompilar, y para poder traducirlo o generar guías impresas en el futuro. Está razonado en [./ARCHITECTURE.md](#).

¿Puedo añadir una dependencia? Con cuidado. Cada plugin es una vía por la que un permiso puede entrar en el manifiesto fusionado sin aparecer en el código, y hay tres controles que bloquearán la publicación si eso ocurre. Hoy hay una dependencia de ejecución.

¿Por qué el metrónomo no usa `Timer.periodic`? Porque acumula el retraso de cada tick y se queda atrás sin recuperarse. Explicado en [06](#), flujo 5.

¿Puedo probar en Android sin dispositivo? Con un emulador, tras ejecutar el bootstrap. Para verificar un APK necesitas además las build-tools con `aapt2`.

¿Qué NO debo hacer nunca? Publicar el estado antes de confirmar la persistencia. Importar Flutter en `lib/domain`. Escribir la versión a mano en cualquier sitio. Añadir cámara, micrófono o red. Relajar un validador para que deje de fallar en vez de corregir lo que señala.

Continuar por

- [04](#) · [Mapa del código](#) cuando busques dónde vive algo.
- [19](#) · [Matriz de trazabilidad](#) para ir de una funcionalidad a su prueba en una fila.